

Iteration and Loop Control Structures in C

[DEF] Definition to Remember

Loops are control structures that repeatedly execute a block of code as long as a specified condition remains true. C supports entry-controlled (for, while) and exit-controlled (do-while) loops.

1. Concept Overview & Fundamentals

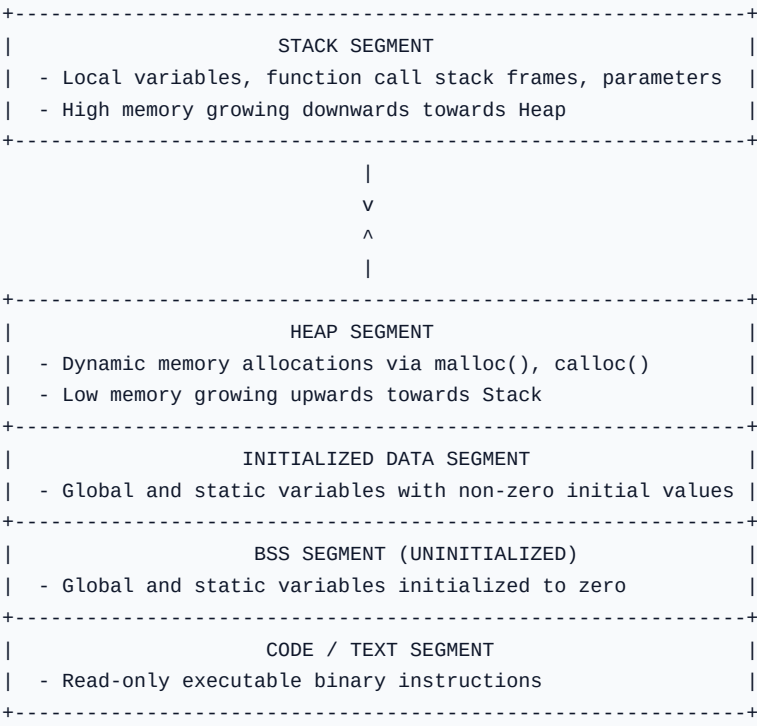
Iteration and Loop Control Structures in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

* Co

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization



Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers	Stack: O(1) push/pop; Heap: O(1)
Type Checking	Static Typing	Compiler verifies types during compila	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memor	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops,	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Iteration & Loop Control Structure\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- * Operating System Kernels: Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

* En

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: ArmstrongNumber

Problem Description & Objective:

C Code Implementation:

Progra

//Prog

#inclu

int ma

/*

*

*

*

*

*

*

*

in

pr

sc

on

wh

```
}  
if  
el  
re  
}
```

Problem 2: EvenPrint

Problem Description & Objective:

2. Pr

C Code Implementation:

```
// 2.  
  
#inclu  
  
int ma  
/*  
*  
*  
*  
*  
*  
*  
*  
*  
  
in  
  
pr  
sc  
  
pr  
  
fo  
  
}  
  
re  
}
```

Problem 3: Factorial

Problem Description & Objective:

Write a

C Code Implementation:

```
//writ  
  
#inclu  
int ma
```

```
/*
 *
 *
 *
 *
 *
 *
 *
 */
pr
in
pr
sc
in
fo
}
pr
re
}
```

Problem 4: Fibonacci

Problem Description & Objective:

C Code Implementation:

Progra

```
//Prog
#include
int ma
/*
 *
 *
 *
 *
 *
 *
 */
in
pr
sc
pr
wh
}
re
}
```

Problem 5: GCD

Problem Description & Objective:

C Code Implementation:

Create

```
//Crea
#include

int ma
/*
*
*
*
*
*
*
*
*
*
*
*
*
in

pr

pr
if

}

pr
if

}

in
in

wh

}

in
pr

re
}
```

Problem 6: InfiniteLoop

Problem Description & Objective:

C Code Implementation:

3. Pr

```
// 3.
#include
```

```
int main()
/*
 *
 *
 *
 *
 *
 */
{
    int n;
    while (n < 0)
    {
        printf("Enter a positive number: ");
        scanf("%d", &n);
    }
}
```

Problem 7: LCM

Problem Description & Objective:

C Code Implementation:

Create

```

//Create
#include
int main
/*
*
*
*
*
*
*
in
pr
pr
so
pr
so
in

```

```
in
fo
}
re
}
```

Problem 8: Multiplication

Problem Description & Objective:

C Code Implementation:

Develo

```
//Deve
#include
int ma
/*
*
*
*
*
*
*
*
pr
in
pr
sc
fo
}
re
}
```

Problem 9: OddSum

Problem Description & Objective:

C Code Implementation:

Create

```
//Crea
#include
int ma
/*
*
*
```

```
*
*
*
*
*
*
in
in
pr
sc
fo
}
pr
re
}
```

Problem 10: PalindromeNumber

Problem Description & Objective:

C Code Implementation:

Progra

```
//Prog
#include
int ma
/*
*
*
*
*
*
*
in
pr
sc
on
wh
}
if
el
re
}
```


Problem 11: PatternPrint

Problem Description & Objective:

C Code Implementation:

Implem

```
#include <stdio.h>

int main()
/*
 *
 *
 *
 *
 *
 *
 */

    int n;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    // Print the pattern
    for (int i = 1; i <= n; i++)
    {
        // Print the pattern for each row
        for (int j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }

    return 0;
}
```

Problem 12: PrimeNumber

Problem Description & Objective:

Program

C Code Implementation:

```
//Program 1.1: A C program to calculate the sum of the first n natural numbers.
#include <stdio.h>

int main()
/*
 *
 *
 *
 *
 *
 */
{
    int n, sum = 0;

    printf("Enter the value of n: ");
    scanf("%d", &n);

    if (n < 1)
    {
        printf("Invalid input. n must be greater than 0.\n");
        return 1;
    }

    for (int i = 1; i <= n; i++)
    {
        sum += i;
    }

    printf("The sum of the first %d natural numbers is: %d\n", n, sum);

    return 0;
}
```

Problem 13: ReverseNumber

Problem Description & Objective:

Program

C Code Implementation:

```
//Prog
#include

int main
    /*
    *
    *
    *
    *
    *
    */
```

```
in
pr
sc

wh

}

pr

re
}
```

Problem 14: SumOfDigit

Problem Description & Objective:

C Code Implementation:

Create

```
//Crea
#include

int ma
/*
*
*
*
*
*
*
*

pr

in

pr
sc

wh

}

pr

re
}
```

Problem 15: SumPositive

Problem Description & Objective:

1. Pr

```
// 1.
// Ski
#include <stdio.h>

int main() {
    // *
    // *
    // *
    // *
    // *
    // *

    int n;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\n");
        for (int j = 0; j < n - i; j++) {
            printf(" ");
        }
        for (int k = 0; k < i + 1; k++) {
            printf("* ");
        }
        printf("\n");
    }
}
```

Problem Description & Objective:

C Code Implementation:

Implement

```

/*
Create
using
*/

#include

int main()
/*
*
*
*
*
*
*
*
in
pr

```

```
fo
}
re
}
```

Problem 17: multiplication table

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
*/

#include

int ma
/*
*
*
*
*
*
*
*

in

pr
sc

pr

fo
}

re
}
```

Problem 18: positive number

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
Use a
the us
*/

#include
```

```
int ma
/*
*
*
for th
*
*
*
*
in
do

}

pr

re
}
```

Problem 19: prime number

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Questi
Create
if a n
*/

#inclu

int ma
/*
*
*
*
*
*
*
*
*
in

pr
if

}

if
```

```
}  
  
}  
if  
}  
}  
re  
}
```

Problem 20: square break loop

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Write  
the us  
  
Use an  
when a  
*/  
  
#inclu  
  
int ma  
/*  
*  
  
infini  
*  
*  
*  
*  
  
in  
wh  
  
}  
pr
```

Problem 21: sum positive continue

Problem Description & Objective:

C Code Implementation:

```
re  
}
```

Implem

```
/*  
Create  
entere  
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
number  
*  
*  
*  
*  
  
in  
  
pr  
  
wh  
  
}  
  
pr  
  
re  
}
```

Problem 22: sum until zero

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Develo  
entere  
The to
```



```
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
total  
*  
*  
*  
*  
in  
do  
  
}  
  
pr  
re  
}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. for loop: Combines initialization, condition test, and update in a single compact header.
2. while loop: Entry-controlled loop; checks condition before executing body (0 to N iterations).
3. do-while loop: Exit-controlled loop; guarantees at least one execution (1 to N iterations).
4. break: Immediately exits the loop; continue: Skips remaining statements in current iteration.
5. Nested loops: Used for multidimensional arrays, matrix arithmetic, and 2D pattern printing.

[RECALL] Quick Recall

Init -> Condition Check -> Loop Body -> Step Update -> Break/Continue Control